

CSE 565 Performance Testing Project

Chase Coleman

Arizona State University

CSE 565: Software Verification and Validation

Performance Testing Project

October 4, 2026

1 Task 1: Identification of the Performance Testing Framework

Locust 2.46.6 was selected as the performance-testing framework. Locust is a Python framework in which a user class defines the work performed by a simulated user. It records request count, failures, response times, response-time percentiles, and requests per second (RPS). The framework can be run from the command line in headless mode and can create HTML and CSV reports. The Locust training module used for the framework setup is *Your first test*: <https://docs.locust.io/en/stable/quickstart.html>.

The instructor-provided `load_test_tasks.py` file was preserved without modification. The separate `locustfile.py` defines a Locust user workflow with the same six workload categories: data processing, simulated database query, API request, file I/O, computation, and logging. A controlled local profile was used so that the 10,000-user test could be executed without directing high-volume traffic to the public API in the starter script.

2 Task 2: Test Case Design

Each Locust user repeatedly executes the six workload categories in one workflow. Locust records a separate **TASK** result for each category. The controlled API task returns a simulated response; the remaining operations perform local processing, waiting, file I/O, computation, and logging. The three baseline cases use the same standard profile and a 30-second observation window.

Table 1: Baseline Locust test cases.

Test	Load level	Users	Spawn rate	Duration	Profile
Test 1	Baseline load	100	100 users/s	30 s	Standard
Test 2	Medium load	1,000	1,000 users/s	30 s	Standard
Test 3	High load	10,000	10,000 users/s	30 s	Standard

The following command is implemented by `run_locust_scenarios.sh`. It invokes Locust in headless mode and saves an HTML report, statistics CSV, statistics-history CSV, failures CSV, exceptions CSV, and console log for each load level.

```
CSE565_PROFILE=standard CSE565_API_MODE=simulated
CSE565_DURATION_SECONDS=30 .venv/bin/locust -f locustfile.py --headless
--users 100 --spawn-rate 100 --stop-timeout 15
--csv locust_results/users-100/locust
--html locust_results/users-100/locust_report.html
```

3 Task 3: Test Execution Results

The three immediate-spawn tests were completed through Locust on October 4, 2026. Table 2 reproduces the aggregate row from each Locust `locust_stats.csv` report. Response-time values are in milliseconds. The result files are retained in the project under `locust_results/20261004-130817/`.

Table 2: Aggregate Locust results for the three baseline tests.

Users	Requests	Failures	Average (ms)	Median (ms)	Maximum (ms)	RPS
100	23,018	0	2.903	0.009	21.449	792.579
1,000	237,133	0	2.804	0.009	49.595	7,895.465
10,000	739,782	0	146.865	0.009	730.365	23,453.975

Table 3: Per-task Locust response-time results.

Load and task	Average (ms)	Median (ms)	RPS
100: Data Processing	0.167	0.057	132.119
100: Database Query	5.618	5.006	132.119
100: API Request	10.981	11.000	132.085
100: File I/O	0.482	0.074	132.085
100: Computation	0.026	0.009	132.085
100: Logging	0.146	0.022	132.085
1,000: Data Processing	0.079	0.057	1,316.771
1,000: Database Query	5.682	5.001	1,316.005
1,000: API Request	10.869	10.002	1,315.672
1,000: File I/O	0.144	0.074	1,315.672
1,000: Computation	0.012	0.009	1,315.672
1,000: Logging	0.040	0.021	1,315.672
10,000: Data Processing	0.062	0.057	4,024.620
10,000: Database Query	435.702	470.000	3,944.789
10,000: API Request	445.580	480.000	3,871.141
10,000: File I/O	0.121	0.071	3,871.141
10,000: Computation	0.010	0.009	3,871.141
10,000: Logging	0.039	0.021	3,871.141

4 Task 4: Analysis of Test Execution Results

The 100-user and 1,000-user immediate-spawn tests completed with zero recorded failures. At 100 users, the aggregate Locust report recorded 792.579 requests per second (RPS), and at 1,000 users it recorded

7,895.465 RPS. The simulated database query and API request had low response times at these two levels. For example, the database-query average was 5.618 ms at 100 users and 5.682 ms at 1,000 users; the API-request average was 10.981 ms and 10.869 ms, respectively.

The 10,000-user test also completed with zero recorded failures. The aggregate RPS increased to 23,453.975, which shows that Locust continued to complete measured task requests. However, the response-time data changed substantially for the simulated database-query and API-request tasks. The database-query average increased to 435.702 ms with a 470.000 ms median and a 729.283 ms maximum. The API-request average increased to 445.580 ms with a 480.000 ms median and a 730.365 ms maximum. In comparison, the data-processing, computation, file-I/O, and logging task averages remained below 1 ms at the same load level.

These results show no reported execution failures, but they do show a substantial responsiveness degradation at 10,000 users for the database-query and API-request portions of the workflow. A response time near one-half second remains a completed response rather than a failure, but it is noticeably slower than the approximately 5–11 ms response times recorded at 100 and 1,000 users. Therefore, the system remained available during the 10,000-user test, while the affected workflow tasks became less responsive.

One limitation is that the workload is executed by the Locust user process itself. Consequently, the increased delay may reflect saturation of the Locust/gevent scheduling process in addition to pressure in the simulated application workload. The results identify the database-query and API-request path as the locations where the test observed degradation, but a separate application target process would be needed before attributing the full response-time increase solely to an application-service bottleneck.

References

Locust. (n.d.). *Your first test*. Locust documentation. Retrieved October 4, 2026, from <https://docs.locust.io/en/stable/quickstart.html>